AI Work-Log

The case expects AI use. I used Claude as a pair-programmer — to explain concepts, draft code/SQL, and debug — while I held the requirements, made the design decisions, wrote parts myself, and validated every output. The value is in where I directed and corrected it, below.

Key prompts (representative)

- "Design the shared event envelope for 3 fictional apps; justify `event_type` vs `event_name`."
- "Build a seeded generator with the full user lifecycle (install→engage→pay→churn), duplicates, malformed payloads, and spikes."
- "I want to write the 7 metric queries myself — review them and catch bugs."
- "Pre-aggregate into a daily summary and prove the before/after with bytes scanned."

Three places I corrected the AI

1. **Real app names.** It proposed `vaultly/storygen/clipzy`. I checked and found **vaultly is a real app** → had it rename to fictional `ledgerly/promptly/ streamio`. I also rejected its suggested `schema_version` field as cargo-culting (I never bump it), keeping only the app `version` the data actually varies.
2. **Generator domain rules.** It let financial events fire before `install`, re-rolled a user's region per event, and allowed refunds/renewals with no prior purchase (plus a renewal inside an impossible 14-day window). I fixed the rules: install is the first event; region is stable per user; refund/renewal require a real purchase; refund only after a 3-day cooling-off. I also caught that app `version` could *decrease* over time (sessions spilled past midnight) and had sessions clamped within the day and made non-overlapping.
3. **"Per app, not per day."** When I asked it to compare my BigQuery results to a Python baseline, its first pass compared only per-app totals. I pushed back — the queries return per-DAY rows — and made it diff every day. That surfaced 2 real install-count differences a per-app view had hidden, which we then correctly traced to the dedup tiebreaker + malformed-data filtering (Python counts the raw file; BigQuery counts what survived ingestion). Money/identity metrics matched exactly.

One prompt that FAILED (wrong output + my fix)

Prompt: "Add a morning-snapshot to each dashboard tile: the latest day's value with a day-over-day change."

Wrong output: For the Revenue tile it rendered:

`Revenue  $0.00  ▲ $79.99  100%`

i.e. it claimed revenue was **up 100%** on a day with **$0** revenue. The AI applied a naïve `latest − previous` to every metric. But revenue is sparse and can be **negative** (refund days): the latest day was $0.00 and the prior day was −$79.99, so `0 − (−79.99) = +79.99` → a green ▲ and a meaningless "100%" measured against a negative base. Correct math, nonsense metric.

My fix: I told it revenue is not a valid day-over-day metric. We changed the revenue tile to a **rolling 7-day total vs the previous 7 days**, and guarded the percentage so it only shows against a **positive** baseline. Dense metrics (active users, feature usage) kept the latest-day delta. The fix is data-aware, not just a formula tweak.

Reflection

The AI was fastest at boilerplate (FastAPI/Express scaffolding, SQL syntax, gcloud) and at explaining concepts on demand. It was **least reliable on domain correctness** — generator lifecycle rules, a metric that ignored refunds, SQL bugs in review — all of which I caught. My job was to hold the requirements, validate every number against an independent source, and reject designs that didn't fit. The live endpoints and performance numbers are real and measured.